Facial Expression Recognition Using Deep Learning

Project Description

The Facial Expression Recognition (FER) system is an intelligent deep learning–based application designed to identify human emotions from facial images or live webcam video. The system analyzes facial features and classifies them into different emotional categories such as angry, disgust, fear, happy, neutral, sad, and surprise.

The application provides an interactive interface where users can either use their webcam for real-time emotion detection or upload images using a drag-and-drop interface. The system detects faces in the input image, preprocesses the detected face, and then uses a trained Convolutional Neural Network (CNN) model to predict the emotion.

The model is trained using the **FER-2013 dataset**, which contains thousands of labeled facial images representing different emotions. By combining **computer vision techniques with deep learning**, the system can analyze facial expressions quickly and accurately.

This project demonstrates the practical application of Artificial Intelligence, Computer Vision, and Deep Learning for emotion recognition and can be extended for applications such as human–computer interaction, mental health monitoring, and smart surveillance systems.

Project Objective

• To detect and classify human emotions using facial expressions.

• To build a deep learning model capable of recognizing emotions from facial images.

• To implement real-time facial expression recognition using a webcam.

• To support drag-and-drop image uploads for emotion detection.

• To preprocess facial images for accurate emotion prediction.

• To deploy the system using web technologies for easy user interaction.

• To demonstrate the use of CNNs in computer vision applications.

Technology Stack

Frontend:
Streamlit / React Web Interface for user interaction

Backend:
Python

Deep Learning Framework:
TensorFlow / Keras

Computer Vision Library:
OpenCV

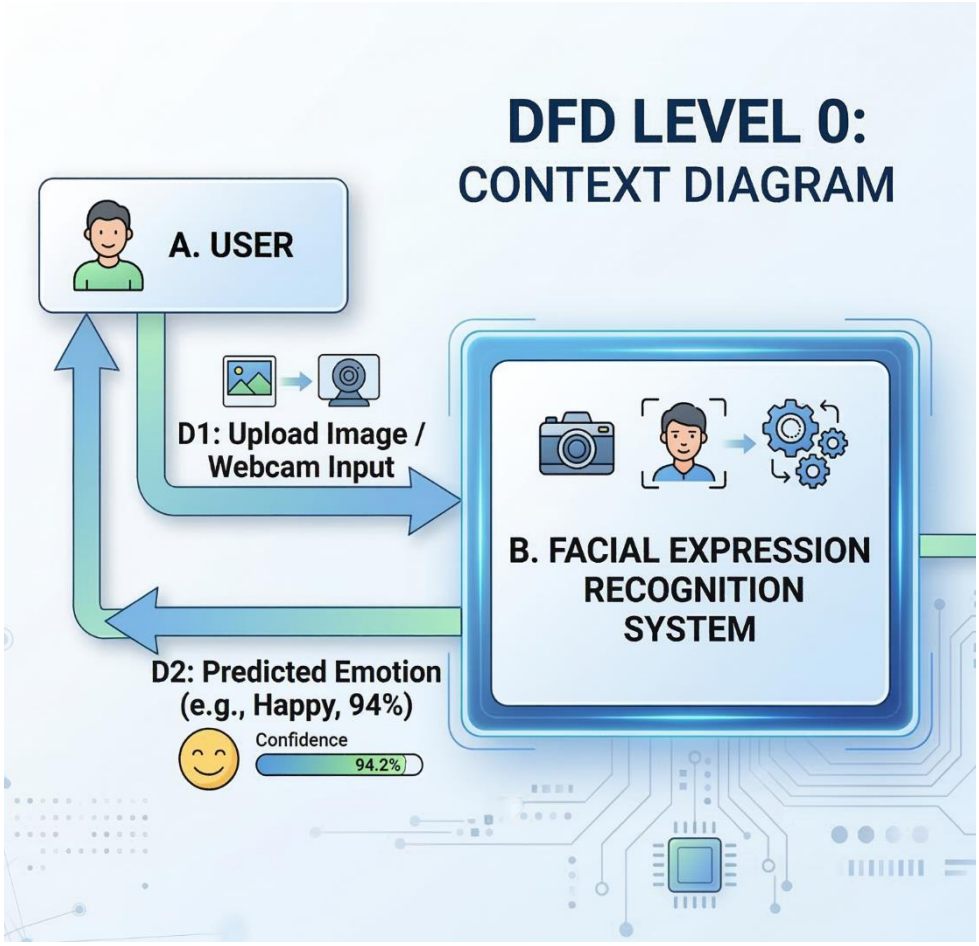
Dataset:
FER-2013 Dataset (Kaggle)

Model File:
emotion_detection.h5

Project DFD

1. DFD (Level 0) – Context Diagram

A[User] -->|Upload Image / Webcam Input| B[Facial Expression Recognition System]
B -->|Predicted Emotion| A



Data Flows

User → System: Facial image or webcam frame input
System → User: Predicted emotion with confidence score

2. DFD (Level 1) – Functional Breakdown

A[User] -->|Input Image / Webcam| B[Input Interface]

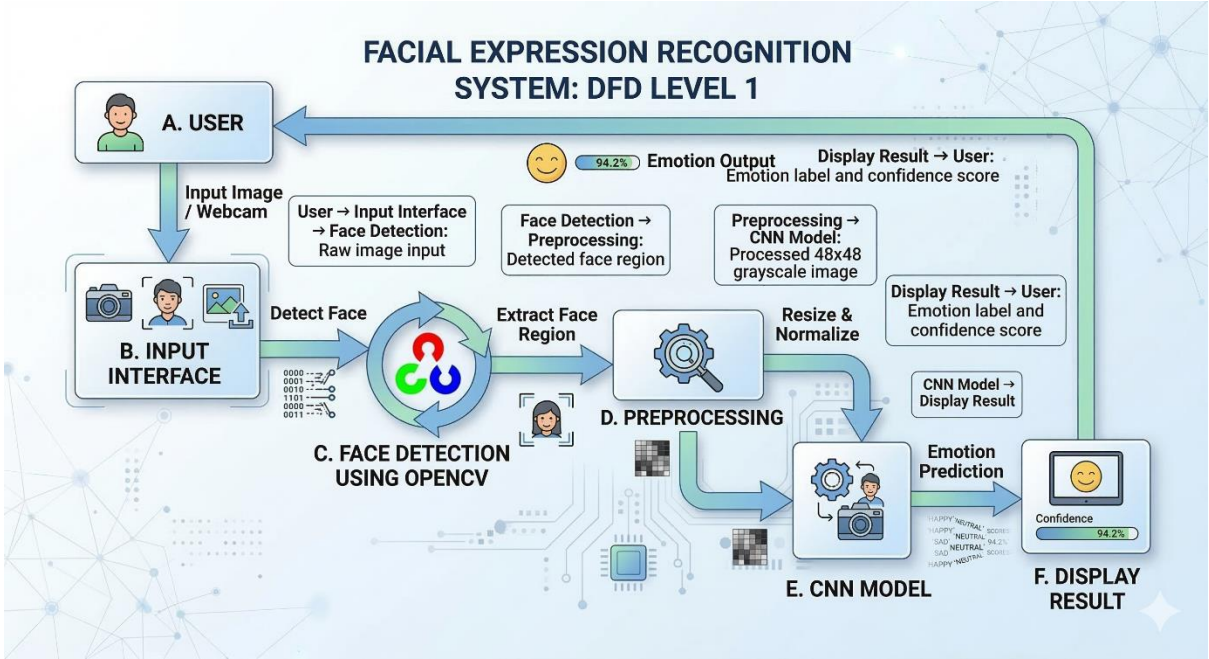
B -->|Detect Face| C[Face Detection using OpenCV]

C -->|Extract Face Region| D[Preprocessing]

D -->|Resize & Normalize| E[CNN Model]

E -->|Emotion Prediction| F[Display Result]

F -->|Emotion Output| A



Data Flows

User → Input Interface → Face Detection: Raw image input

Face Detection → Preprocessing: Detected face region

Preprocessing → CNN Model: Processed 48x48 grayscale image

CNN Model → Display Result: Emotion prediction

Display Result → User: Emotion label and confidence score

Folder Structure

```
FER_Project/
│
├── emotion-ui/
├── fer_env/
├── node_modules/
├── test/
├── tf213_env/
├── train/
│
├── .gitignore
├── app.py
├── convert_model.py
├── dataset.zip
├── emotion_model.h5
├── package.json
├── README.md
├── real_time_demo.py
├── requirements.txt
├── runtime.txt
├── streamlit_app.py
├── train_model.py
```

SOURCE CODE

App.py

app.py > ...

```python
from flask import Flask, request, jsonify
from flask_cors import CORS
import base64
import numpy as np
import cv2
from tensorflow.keras.models import load_model

import os
app = Flask(__name__)
CORS(app)

# Load trained model


model = load_model("emotion_model.h5")
emotion_labels = ['angry','disgust','fear','happy','neutral','sad','surp

# Load face detector
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
)

@app.route("/")
def home():
    return "Emotion Detection API Running"

@app.route("/detect", methods=["POST"])
def detect():
    try:
        data = request.json

```

```python
def detect():
        return jsonify({"emotion": "No Image Received", "confidence

        image_data = data["image"]

        # Decode base64 image
        image_bytes = base64.b64decode(image_data.split(",")[1])
        np_arr = np.frombuffer(image_bytes, np.uint8)
        img = cv2.imdecode(np_arr, cv2.IMREAD_COLOR)

        # Fix mirror webcam issue
        img = cv2.flip(img, 1)

        # Convert to grayscale
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

        # Improve contrast
        gray = cv2.equalizeHist(gray)

        faces = face_cascade.detectMultiScale(
            gray,
            scaleFactor=1.3,
            minNeighbors=5,
            minSize=(30,30)
        )

        if len(faces) == 0:
            return jsonify({"emotion": "No Face Detected", "confidence":

        # Use first detected face
        (x, y, w, h) = faces[0]
```

Model Training (train_model.py)

train_model.py 5 × runtime.txt README.md real_time_demo.py

train_model.py > ...

```python
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dense, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
import matplotlib.pyplot as plt

# ========================
# SETTINGS
# ========================
img_size = 48
batch_size = 32

# ========================
# DATA GENERATORS
# ========================

train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=10,
    zoom_range=0.1,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
)

test_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(
    'train',
```

```python
# ========================
# BUILD STRONG CNN MODEL
# ========================

model = Sequential()

model.add(Conv2D(64, (3,3), activation='relu', input_shape=(48,48,1)))
model.add(BatchNormalization())
model.add(MaxPooling2D(2,2))
model.add(Dropout(0.25))

model.add(Conv2D(128, (3,3), activation='relu'))
model.add(BatchNormalization())
model.add(MaxPooling2D(2,2))
model.add(Dropout(0.25))

model.add(Conv2D(256, (3,3), activation='relu'))
model.add(BatchNormalization())
model.add(MaxPooling2D(2,2))
model.add(Dropout(0.25))

model.add(Flatten())

model.add(Dense(256, activation='relu'))
model.add(Dropout(0.5))

model.add(Dense(train_generator.num_classes, activation='softmax'))
```

.gitignore

.gitignore

```
# Node / React
node_modules/
npm-debug.log*
yarn-debug.log*
yarn-error.log*

# Build folders
dist/
build/

# OS files
.DS_Store
Thumbs.db

# VS Code
.vscode/

# Logs
*.log

# Temporary files
*.tmp
*.temp

train/
test/
dataset.zip
*.h5
```

requirements.txt

requirements.txt

```
streamlit
tensorflow==2.20.0
opencv-python-headless
numpy
Pillow
```

index.html

index.html M × | favicon.ico | requirements.txt | .gitignore | ImageEmot

emotion-ui > public > index.html > html > head > meta

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <link rel="icon" href="%PUBLIC_URL%/favicon.ico" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <meta name="theme-color" content="#00003" />
    <meta
      name="description"
      content="Web site created using create-react-app"
    />
    <link rel="apple-touch-icon" href="%PUBLIC_URL%/logo192.png" />
    <!--
      manifest.json provides metadata used when your web app is installed on
      user's mobile device or desktop. See https://developers.google.com/web/
    -->
    <link rel="manifest" href="%PUBLIC_URL%/manifest.json" />
    <!--
      Notice the use of %PUBLIC_URL% in the tags above.
      It will be replaced with the URL of the `public` folder during the buil
      Only files inside the `public` folder can be referenced from the HTML.

      Unlike "/favicon.ico" or "favicon.ico", "%PUBLIC_URL%/favicon.ico" will
      work correctly both with client-side routing and a non-root public URL.
      Learn how to configure a non-root public URL by running `npm run build`
    -->
    <title>React App</title>
  </head>
  <body>
    <noscript>You need to enable JavaScript to run this app.</noscript>
    <div id="root"></div>
```

Streamlit_app.py

streamlit_app.py > ...

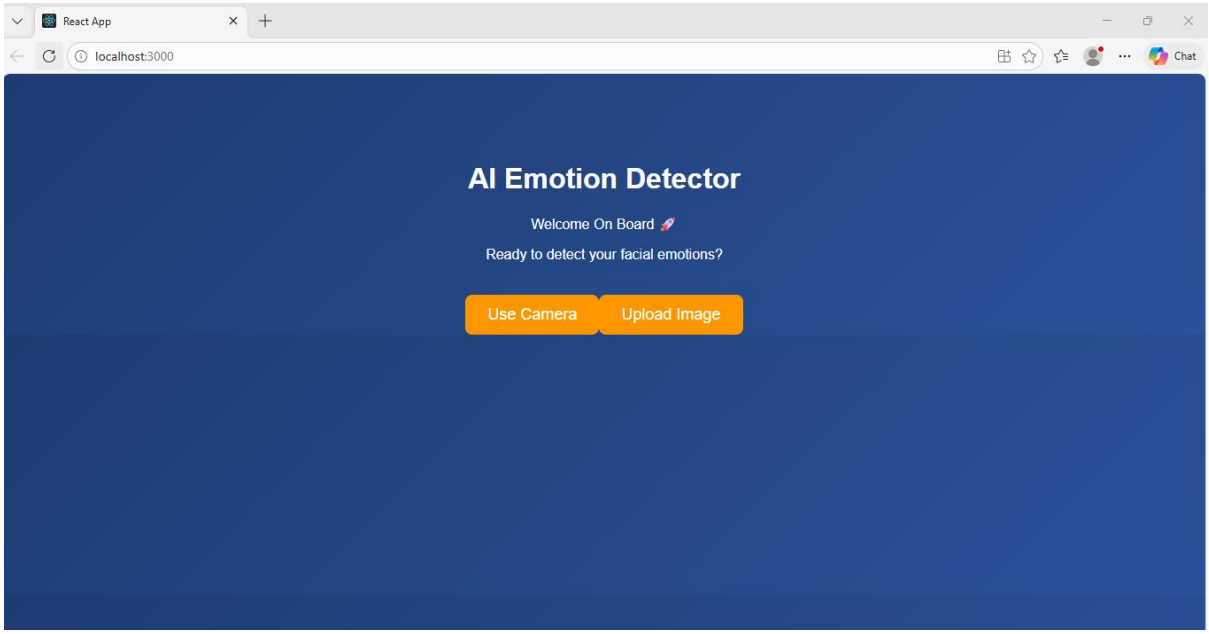
```python
import streamlit as st
import numpy as np
import cv2
from tensorflow.keras.models import load_model
from PIL import Image

# Load model


model = load_model("emotion_model_tf213.h5")
emotion_labels = [
    "Angry",
    "Disgust",
    "Fear",
    "Happy",
    "Sad",
    "Surprise",
    "Neutral"
]

st.title("AI Facial Emotion Detection")

uploaded_file = st.file_uploader("Upload an image", type=["jpg","png","jp

if uploaded_file is not None:

    image = Image.open(uploaded_file)
    st.image(image, caption="Uploaded Image", use_column_width=True)

    img = np.array(image)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

streamlit_app.py > ...

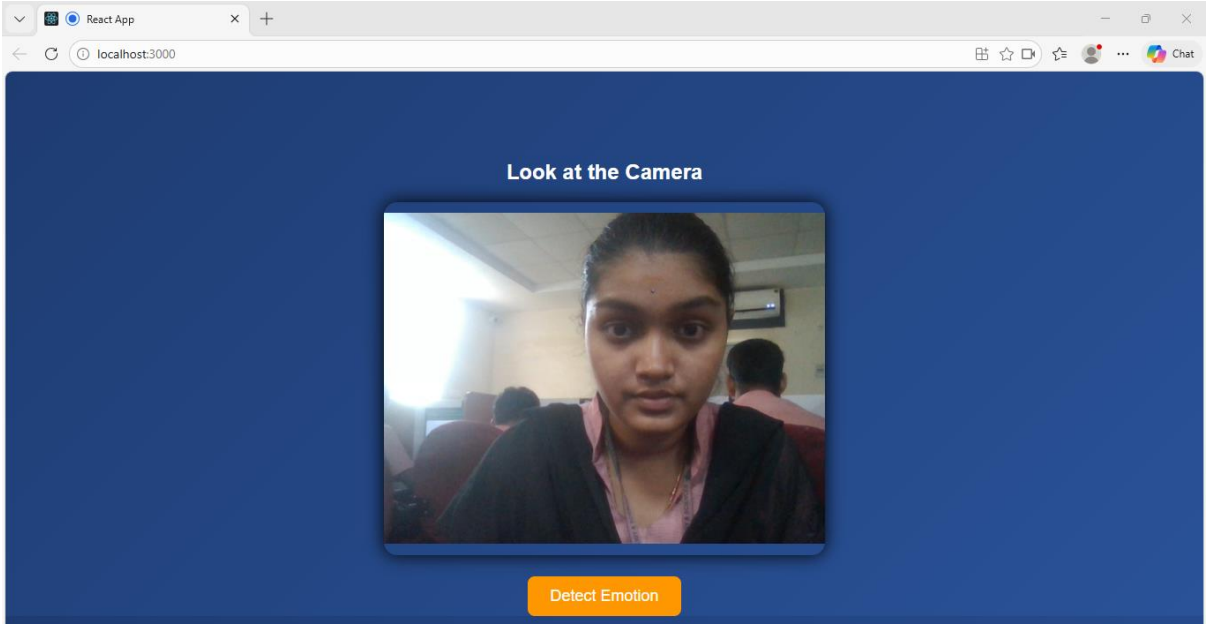
```python

    face = cv2.resize(gray, (48,48))
    face = face/255.0
    face = face.reshape(1,48,48,1)

    prediction = model.predict(face)
    emotion = emotion_labels[np.argmax(prediction)]

    st.subheader(f"Detected Emotion: {emotion}")
```

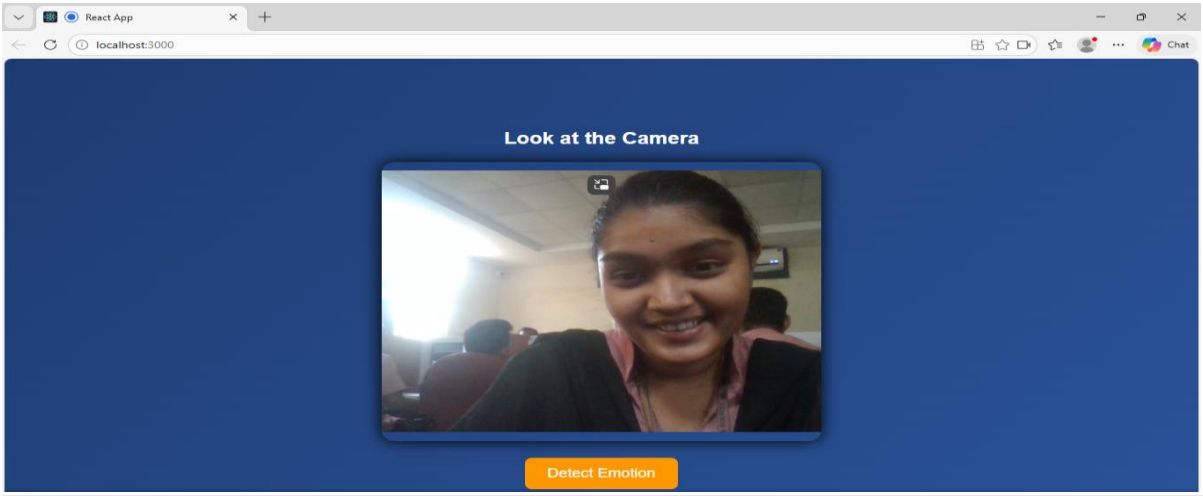
Output Screenshots

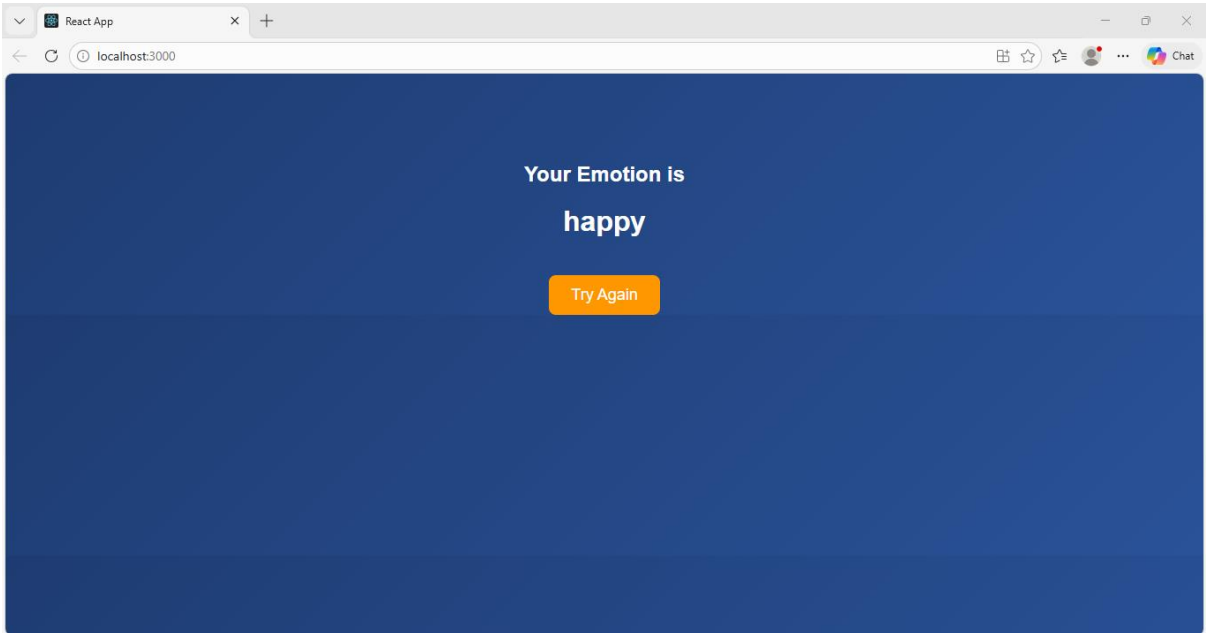
Home Page

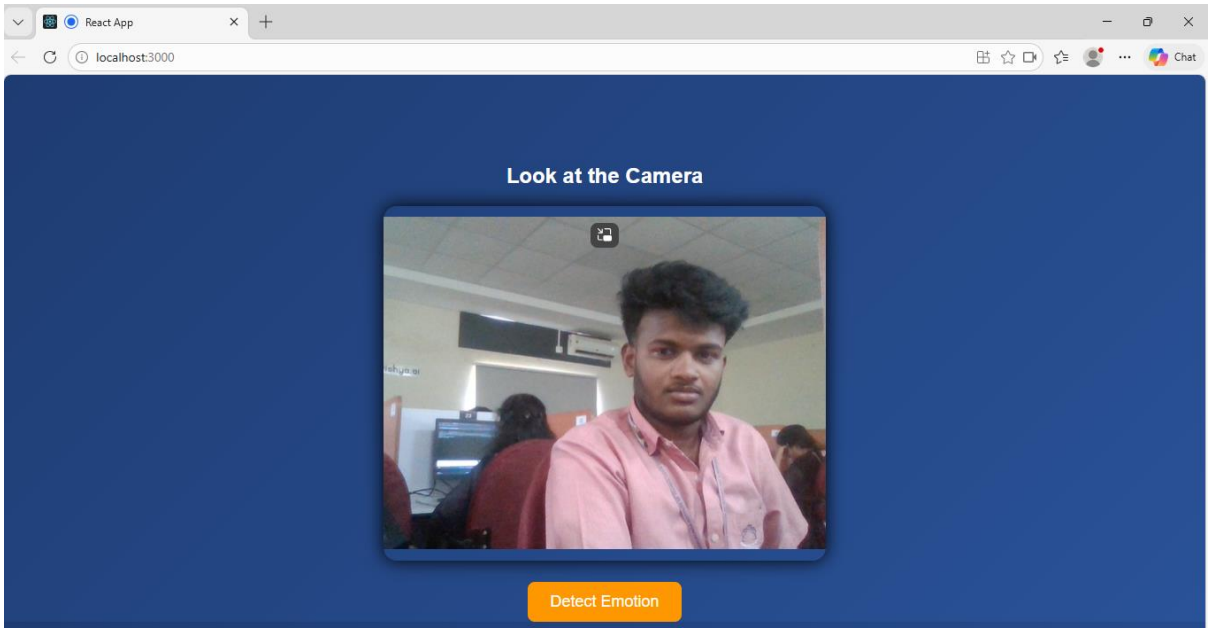


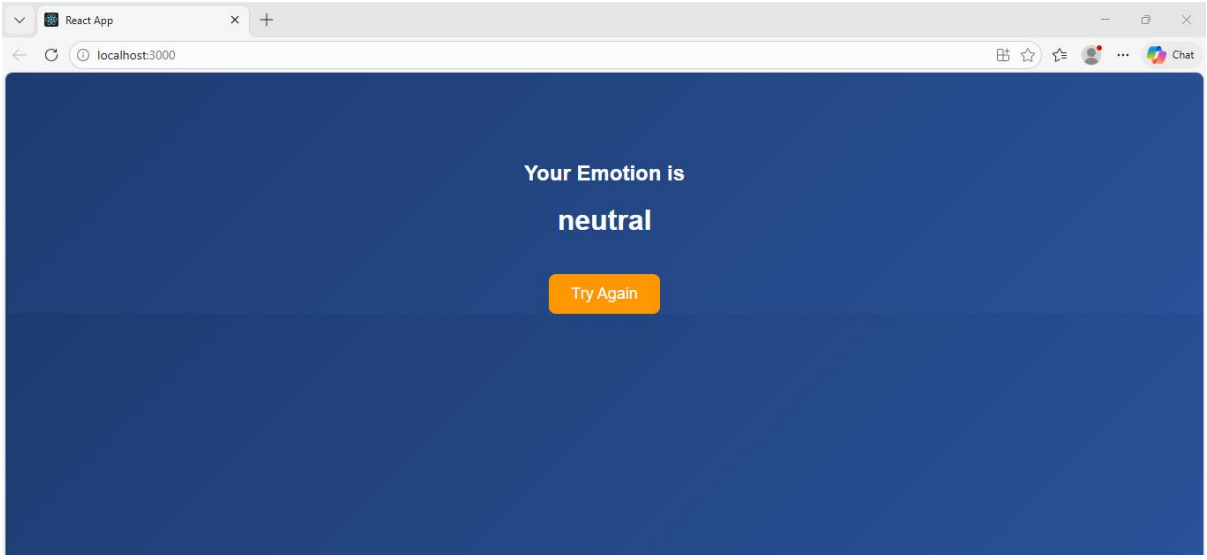
Web cam input

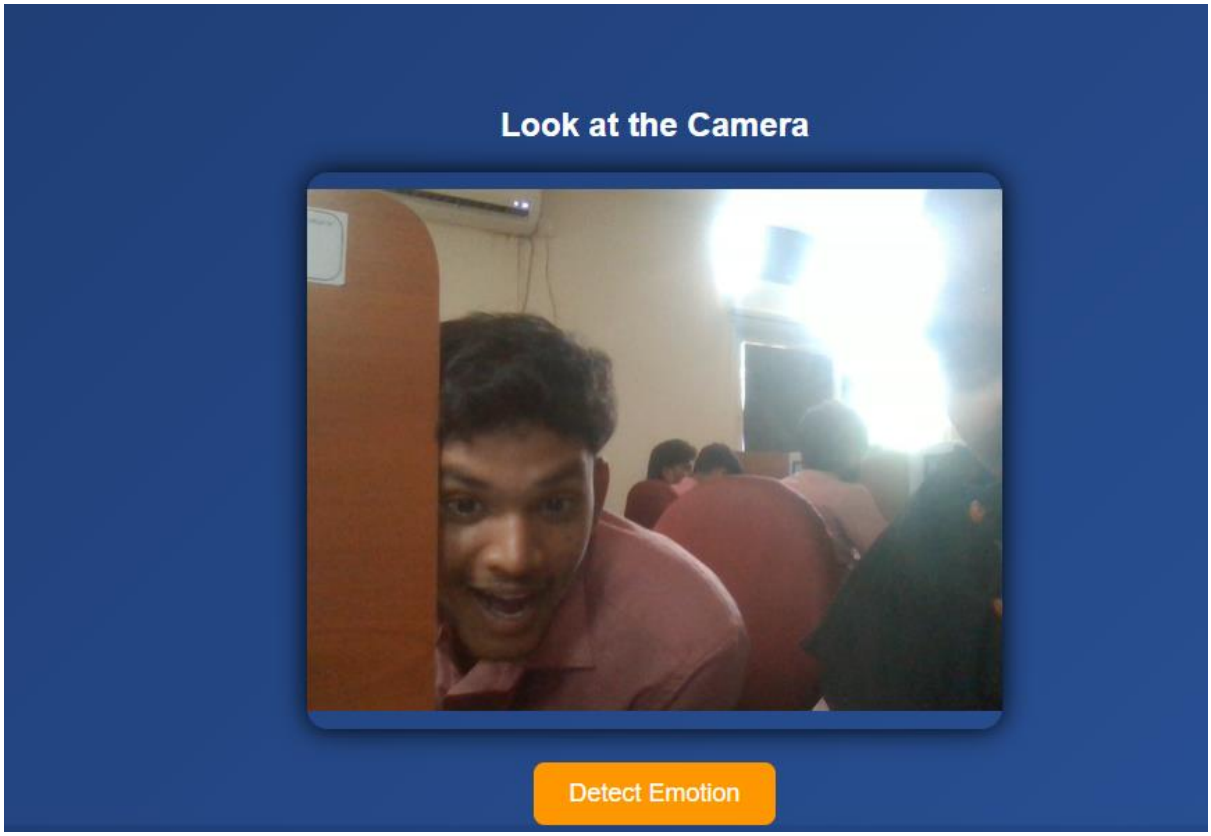


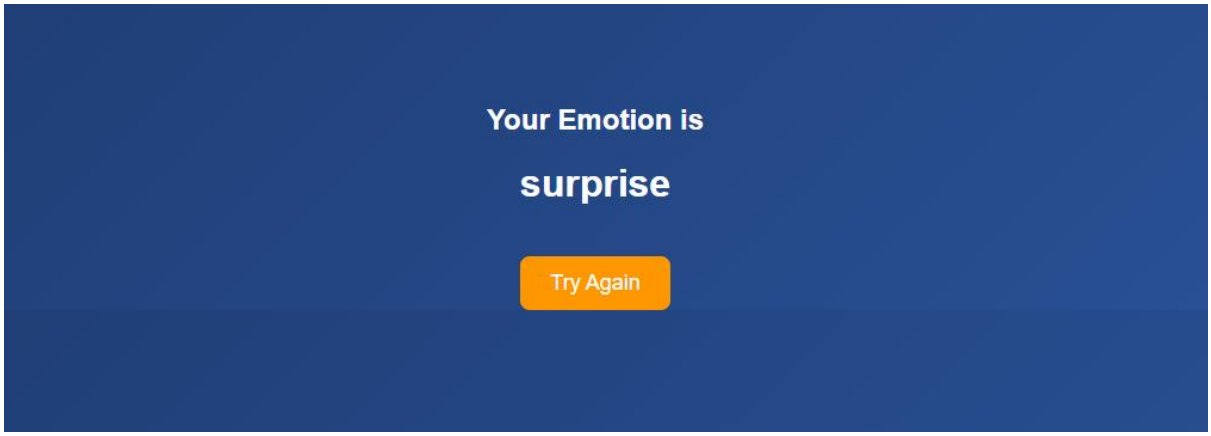




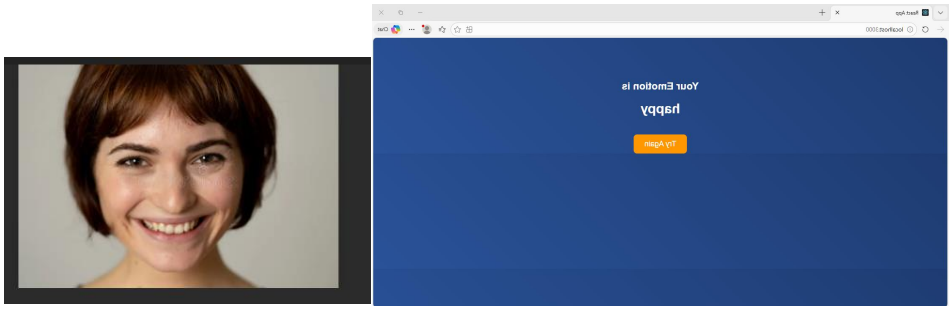

React App
localhost:3000
Chat
Look at the Camera
Detect Emotion

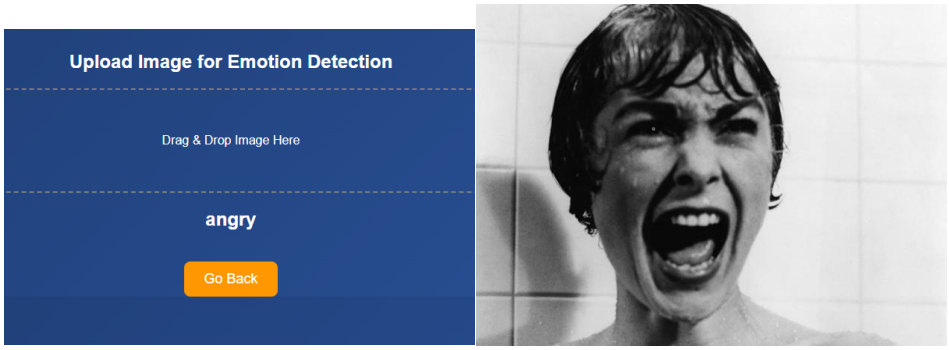

React App
localhost:3000
Chat
Your Emotion is
neutral
Try Again

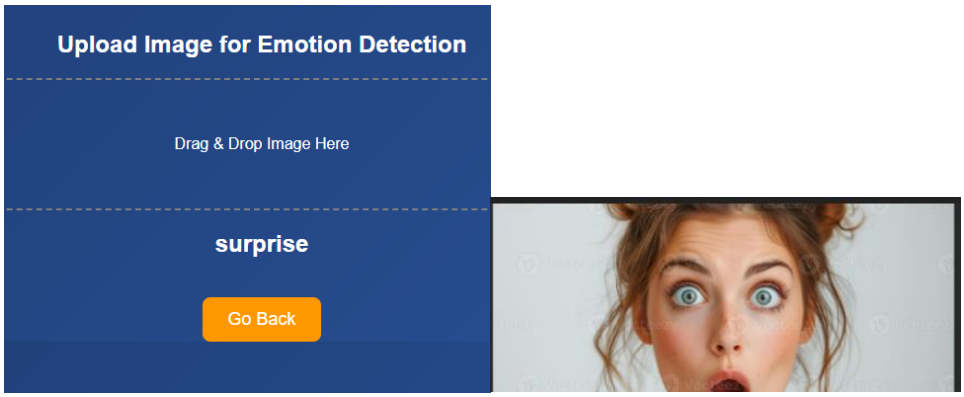

Look at the Camera
Detect Emotion

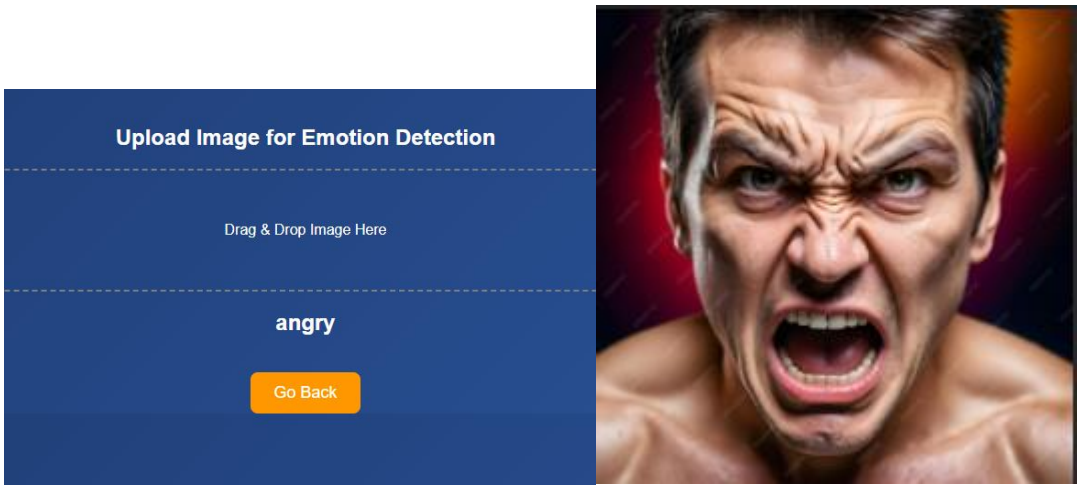

Your Emotion is
surprise
Try Again

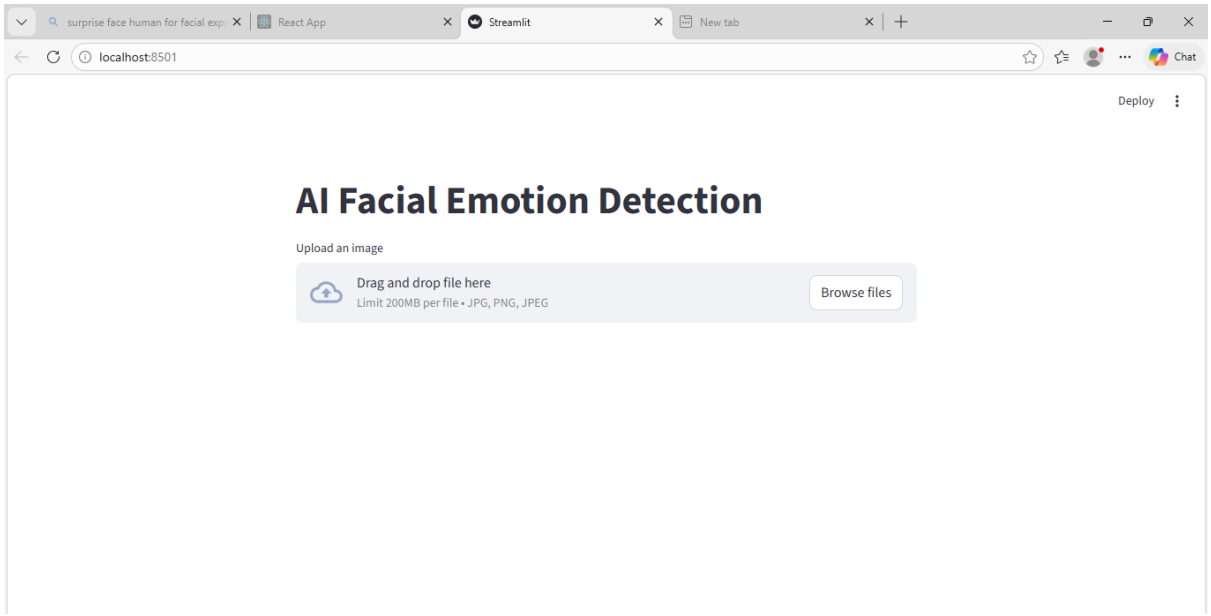
Drag and Drop Option

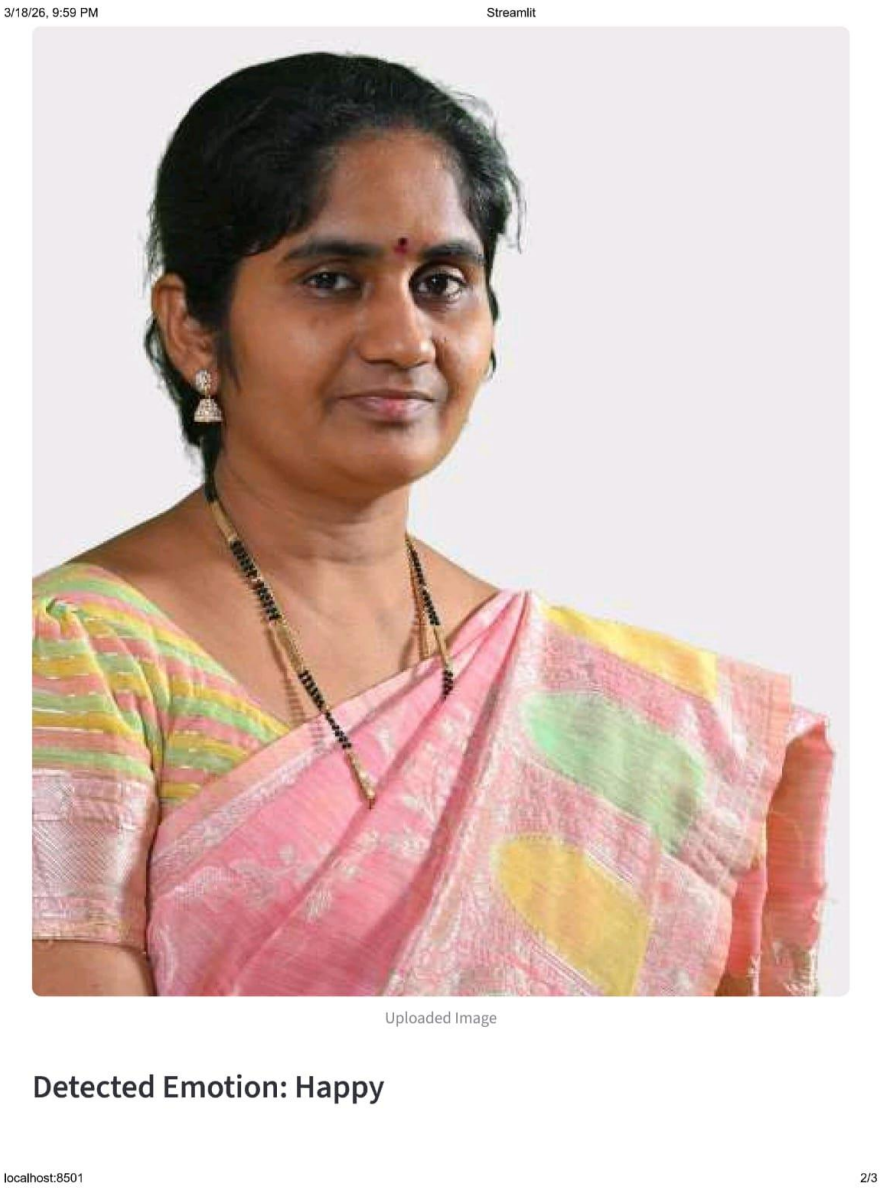









surprise face human for facial expr
React App
Streamlit
New tab
localhost:8501
Chat
Deploy
AI Facial Emotion Detection
Upload an image
Drag and drop file here
Limit 200MB per file • JPG, PNG, JPEG
Browse files


3/18/26, 9:59 PM
Streamlit
Uploaded Image
Detected Emotion: Happy
localhost:8501
2/3